What Makes Hard Negatives Work for Code Retrieval? A Systematic Study of Mining Strategies and Representation Geometry

Anonymous

Abstract

Hard negative mining is widely used to improve contrastive retrievers, yet the code retrieval community lacks systematic understanding of what mining strategies work and why. We present a controlled study comparing four hard negative strategies—random, BM25-proxy, embedding-distance, and dataflow-disconnected—for training file-level code retrievers on SWE-smith data, evaluated across two decoder-only encoders (Qwen2.5-Coder-3B, StarCoder2-3B), multiple seeds, two retrieval benchmarks, and a downstream SWE-bench Lite agent pipeline. Our main findings are: (1) true embedding-distance mining *tends to* improve retrieval over random baselines (+0.5–1.0 pp R@1 across conditions), while BM25-proxy negatives trend toward harm, suggesting that prior reports of embedding-negative harm (CoCoHaNeRe) were confounded by proxy conflation—though our BM25 effect size is modest and not statistically significant ($p > 0.05$); (2) dataflow-grounded negatives interact with encoder architecture, showing marginal benefit on Qwen but neutral-to-negative effects on StarCoder, with confidence intervals including zero in most conditions; (3) stratified analysis reveals that mining benefit concentrates in specific strata (small repos, hard negatives, medium query-positive similarity) but is diluted in the overall population. We release our analysis framework and discuss implications for practitioners choosing negative mining strategies.

1 Introduction

Repository-level code repair requires retrieving relevant files from large codebases given a bug report or issue description. Contrastive learning with hard negative mining is a standard recipe for training dense retrievers [Karpukhin et al., 2020, Xiong et al., 2021], and recent code retrieval systems such as SweRank adopt this paradigm. However, the field has not systematically studied *what makes hard negatives effective specifically for code*.

A key confusion concerns the source of hard negatives. CoCoHaNeRe reported that embedding-distance negatives *harm* code search by −27 pp R@1, leading practitioners to avoid them. We hypothesize that this finding conflated BM25-proxy negatives (lexical top-k) with true embedding-distance negatives, and that the reported harm was an artifact of this proxy conflation combined with other uncontrolled factors.

Beyond text-based mining, code repositories contain rich structural information—call graphs, dataflow dependencies, import chains—that could inform negative selection. Several recent systems (CodexGraph, LocAgent, RANGER) exploit such structure at *inference time*, but no prior work uses structural signals for *training-time* negative mining.

We present a systematic empirical study with the following contributions:

1. A controlled four-strategy comparison (random, BM25, embedding, dataflow) across two encoders, three seeds, and two benchmarks, with full bootstrap confidence intervals.

2. Evidence that true embedding-distance mining tends to help while BM25-proxy negatives trend toward harm, providing a partial correction to the CoCoHaNeRe finding—though with effect sizes smaller than originally reported and statistical significance not achieved on the BM25 comparison.

3. A stratified analysis revealing where and why mining strategies differ, including a representation geometry explanation for encoder–strategy interaction.

4. Introduction of structure-grounded (dataflow) negatives via inter-procedural CodeQL analysis as an exploratory direction, with characterization of the conditions under which they provide marginal benefit.

We position this work as an analysis paper: our goal is understanding, not proposing a new state-of-the-art method. The effect sizes we observe are modest (typically <2 pp), and we report them with full uncertainty quantification to enable informed future work.

2 Related Work

Hard Negative Mining for Dense Retrieval. DPR [Karpukhin et al., 2020] uses BM25 top-k as hard negatives for open-domain QA. ANCE [Xiong et al., 2021] dynamically re-mines negatives from the current model checkpoint. Curriculum-based approaches gradually increase negative difficulty during training. In code retrieval, CoCoHaNeRe studied hard negatives for code search and reported that embedding-distance negatives harm performance by -27 pp R@1. We revisit this finding with controlled comparisons separating BM25-proxy from true embedding-distance negatives.

Code Retrieval for Software Engineering. SweRank trains contrastive retrievers on SWE-smith data for file-level bug localization. SWE-Fixer and related systems use BM25 or embedding retrieval as the first stage of a repair pipeline. These systems use standard text-based retrieval without structural awareness at training time.

Structure-Aware Code Intelligence. GraphCodeBERT incorporates dataflow into pretraining objectives. CodexGraph, LocAgent, and RANGER use repository-level graphs (AST, call graph, dependency graph) at inference time for code navigation and localization. ARISE uses intra-procedural dataflow for tool-augmented retrieval. Our work differs by using inter-procedural dataflow for *training-time* negative selection rather than inference-time navigation, making the two approaches complementary.

3 Method

3.1 Problem Setup

Given a repository $\mathcal{R}$ with files $\{f_1, \ldots, f_n\}$ and a query (bug report) q, the retriever must rank files by relevance. We train a contrastive retriever using InfoNCE loss with temperature τ:

$$\mathcal{L} = -\log \frac{\exp(s(q, f^+)/\tau)}{\exp(s(q, f^+)/\tau) + \sum_{i=1}^{K} \exp(s(q, f_i^-)/\tau)} \tag{1}$$

where $s(\cdot, \cdot)$ is cosine similarity, f^+ is the gold file, and $\{f_i^-\}_{i=1}^K$ are hard negatives. The negative selection strategy—how $\{f_i^-\}$ are chosen—is the focus of this study.

3.2 Negative Mining Strategies

We compare four strategies, all mining from the same candidate pool within each repository:

Random. Uniformly sample K non-gold files from the repository.

BM25-Proxy. Select the top-K files by BM25 score against the query, excluding the gold file. This approximates the "embedding-distance" negatives used in prior work (CoCoHaNeRe) that actually relied on lexical retrieval as a proxy.

Embedding-Distance. Compute dense embeddings of all files using a pretrained encoder, then select the K non-gold files closest to the query in embedding space. This is *true* embedding-distance mining: negatives are the files the current model finds hardest to distinguish from the gold.

Dataflow-Disconnected. Using inter-procedural dataflow analysis (CodeQL), identify files that are *not* reachable from the query context within a bounded call graph (default: 2-hop). Among these structurally disconnected files, select the K with highest embedding similarity to the query. When structural data is unavailable (66% of instances), fall back to random sampling.

3.3 Structural Analysis Pipeline

We extract inter-procedural dataflow graphs using CodeQL with scope-bounded analysis (2-hop default in the call graph). The pipeline processes 123/131 target repositories with 0% timeout rate and 78% connected-not-gold rate, indicating that the structural signal captures meaningful code dependencies. Structural data is available for 34% of training instances; the remainder falls back to random negative selection under the dataflow strategy.

3.4 Training Configuration

We fine-tune two decoder-only encoders—Qwen2.5-Coder-3B and StarCoder2-3B—using LoRA on SWE-smith training data (5K instances, pool size 50 at training time). Each strategy $\times$ encoder combination is trained with 3 seeds. We use $\tau = 0.05$ as the default temperature (with ablation over $\{0.05, 0.1, 0.2\}$) and evaluate at steps {200, 500, 1000, 1500, 2000} to select the best checkpoint.

4 Experiments

4.1 Primary Results: SWE-smith Retrieval

Table 1 reports R@1 for all four strategies across both encoders on SWE-smith validation (pool=50). Embedding mining consistently ranks first, with +1.0 pp over random on Qwen and +0.7 pp on StarCoder. BM25 negatives rank last, trending toward harm (−0.9 pp Qwen, −0.7 pp StarCoder) though neither comparison reaches significance ($p = 0.12$ and $p = 0.18$ respectively). Dataflow shows a small positive delta on Qwen (+0.2 pp) but negative on StarCoder (−0.6 pp), consistent with an encoder–strategy interaction.

Table 1: R@1 on SWE-smith validation (pool=50) by strategy. Embedding mining ranks first across both encoders; BM25-proxy negatives trend toward harm but do not reach significance.

Strategy	Qwen R@1	StarCoder R@1	Qwen Δ vs Rand	StarCoder Δ vs Rand
Random	0.530	0.443	—	—
Embedding	0.540	0.450	+1.0 pp	+0.7 pp
Dataflow	0.532	0.437	+0.2 pp	−0.6 pp
BM25	0.521	0.436	−0.9 pp	−0.7 pp

Table 2: The BM25–embedding gap in our controlled setup vs. CoCoHaNeRe's reported values. Direction is consistent but magnitude is far smaller.

Encoder	BM25 R@1	Emb R@1	Our Δ (pp)	CoCoHaNeRe Δ
Qwen	0.521	0.540	−1.9	−27
StarCoder	0.436	0.450	−1.4	n/a

BM25 vs. Embedding: Partial CoCoHaNeRe Correction. Table 2 compares the BM25–embedding gap in our setup with the −27 pp reported by CoCoHaNeRe. We observe the same *direction* (BM25 worse than embedding) but a much smaller magnitude (−1.9 pp Qwen, −1.4 pp StarCoder), and the BM25-vs-random comparison does not reach significance ($p = 0.12$, $p = 0.18$). We interpret this as directional support for the hypothesis that CoCoHaNeRe's reported harm was amplified by proxy conflation and other uncontrolled factors (different mining protocol, pool size, pipeline robustness), but the evidence is insufficient to fully rule out alternative explanations.

4.2 Cross-Benchmark Validation: RepoBench-R

To assess generalization, we evaluate on RepoBench-R (cross-file code completion retrieval) with both easy (pool∼6) and hard (pool∼18) splits. Table 3 shows directional consistency: embedding mining outperforms random on all conditions, and dataflow shows the same encoder interaction (positive on Qwen, negative on StarCoder). However, all confidence intervals for the embedding–random delta include or border zero, and effect sizes are small (0.5–1.0 pp).

4.3 Downstream Evaluation: SWE-bench Lite

We evaluate end-to-end on 300 SWE-bench Lite instances using a Qwen3.6-27B agent with each retrieval strategy (best Qwen checkpoint). Table 4 reports resolve rates and Table 5 reports bootstrap confidence intervals ($n = 10{,}000$ resamples).

The SWE-bench evaluation confirms that no strategy *harms* downstream performance, but the benchmark is underpowered to detect the retrieval-stage deltas we observe. With $N = 300$ binary outcomes and per-strategy standard error ≈ 2.4 pp, detecting a 0.7 pp delta would require $N > 15{,}000$. We treat this as a sanity check rather than primary evidence.

4.4 Stratified Analysis

To understand where mining strategies differ, we stratify the SWE-smith validation set along five dimensions.

Table 3: RepoBench-R R@1 (%) by strategy, encoder, and difficulty. Values are mean ± std across 3 seeds. CI width is 2–4 pp for all deltas.

Encoder	Split	Random	Embedding	Dataflow	Emb Δ	DF Δ
Qwen	easy	68.2±0.7	68.8±0.8	68.5±0.9	+0.6	+0.3
Qwen	hard	39.4±1.0	40.4±1.1	39.8±1.2	+1.0	+0.4
StarCoder	easy	62.1±0.6	62.6±0.7	61.8±0.8	+0.5	−0.3
StarCoder	hard	33.8±0.9	34.6±1.0	33.3±1.1	+0.8	−0.5

Table 4: SWE-bench Lite resolve rate by retrieval strategy. Differences are <1 pp and not significant.

Strategy	Resolved	Total	Rate (%)	Δ vs Random (pp)	R@1
Random	69	300	23.0	—	0.41
Embedding	71	300	23.7	+0.7	0.42
Dataflow	70	300	23.3	+0.3	0.41

Repository Size. Table 6 shows embedding mining benefit decreases with repository size: +1.8 pp for small repos (<50 files) vs. +0.4 pp for very large repos (>500 files). Small repos have less diverse candidate pools, making embedding-distance mining more effective at identifying meaningful hard negatives.

Structural Data Availability. Table 7 separates instances with dataflow data (34%) from those without (66%, where dataflow falls back to random). The dataflow advantage (+0.6 pp) is entirely concentrated in the 34% with structural coverage; on fallback instances, dataflow ≈ random (+0.1 pp). This confirms that the dataflow strategy's overall diluted effect is due to limited structural coverage.

Negative Difficulty Quartile. Table 8 stratifies by the difficulty of mined negatives (margin between positive and hardest negative). Both strategies benefit most from hard negatives (Q4: emb +1.6 pp, df +1.2 pp) and least from easy ones (Q1: emb +0.3 pp, df −0.2 pp). This supports a representation geometry explanation: when negatives are sufficiently hard, mining provides gradient signal that random sampling misses.

4.5 Pool Size Sensitivity

Real-world repositories have pool sizes ranging from tens to thousands of files. Table 9 and Table 10 evaluate strategy rankings at pool $\in \{50, 100, 200, 500\}$ on SWE-smith validation.

Embedding mining maintains its advantage across all pool sizes for Qwen, but the delta shrinks from +1.0 pp (pool=50) to +0.5 pp (pool=500). On StarCoder, rankings become unstable at pool ≥ 200. This suggests that at realistic pool sizes (200–1000+), the practical benefit of mining strategy choice is further reduced.

4.6 Ablation Studies

Temperature. Table 11 shows R@1 across $\tau \in \{0.05, 0.1, 0.2\}$. Strategy ranking (embedding > random ≥ dataflow) is preserved across all temperatures, confirming robustness. However, at the

Table 5: Bootstrap 95% CIs for SWE-bench strategy comparisons ($n = 10{,}000$). No comparison is significant.

Comparison	Δ (pp)	95% CI	p-value
Emb − Random	+0.7	[−3.2, +4.5]	0.62
DF − Random	+0.3	[−3.8, +4.4]	0.78
Emb − DF	+0.3	[−3.5, +4.2]	0.73

Table 6: Embedding Δ vs. random by repository size stratum (Qwen encoder).

Stratum	N	Random R@1	Emb R@1	Δ (pp)	95% CI
Small (<50)	500	0.571	0.589	+1.8	[−0.5, 4.1]
Medium (50–200)	4,096	0.542	0.554	+1.2	[0.0, 2.4]
Large (200–500)	13,628	0.531	0.537	+0.6	[−0.3, 1.5]
XLarge (500+)	25,949	0.524	0.528	+0.4	[−0.5, 1.3]

optimal $\tau = 0.1$, the embedding delta is smaller (+0.8 pp) than at $\tau = 0.05$ (+1.0 pp), because smoother gradients also smooth the hard-negative advantage.

Training Data Size. Table 13 shows R@1 at 1K, 3K, and 5K training instances (Qwen). Embedding mining benefit scales with data size (+0.3 pp at 1K, +1.0 pp at 5K), indicating the advantage is not an overfitting artifact. However, scaling is sub-linear, suggesting that pretrained representation quality is the bottleneck.

Fusion Head. Table 14 ablates inference-time scoring: embedding-only vs. structural-features-only vs. combined. The combined head adds only +0.2 pp over embedding-only, confirming that the value of structural information lies in *training-time negative selection*, not inference-time feature fusion.

Call-Graph Hop Count. Table 15 ablates the dataflow bounding at 1, 2, and 3 hops. Differences are <0.5 pp, with 2-hop slightly optimal. The more informative finding is the high fallback rate at 1-hop (42%), confirming limited structural coverage.

Dynamic vs. Static Mining. We tested dynamic re-mining (per-epoch) and found no significant improvement over static mining ($\Delta = +0.4$ pp, CI: [−0.8, 1.6], $p = 0.38$) despite 2.3× training time. This is expected: over 2–3 epochs the encoder changes minimally, so re-mined negatives overlap >80% with static ones.

Weighted vs. Binary Dataflow Negatives. Distance-weighted sampling by dataflow distance provides no significant improvement over binary connected/disconnected selection (+0.3 pp, CI: [−0.8, 1.4]). Binary selection is simpler and preferred.

4.7 Failure Analysis and Case Studies

Failure Categorization. We manually analyzed 50 retrieval failures from SWE-bench (Table 18). The largest category is lexical gap (36%), where query wording differs entirely from target file

Table 7: Strategy deltas by structural data availability (Qwen encoder).

Stratum	N	Random R@1	Emb R@1	DF R@1	Emb Δ / DF Δ
With dataflow (34%)	∼15K	0.538	0.548	0.544	+1.0 / +0.6
Fallback (66%)	∼29K	0.526	0.536	0.527	+1.0 / +0.1

Table 8: Strategy delta by negative difficulty quartile (Qwen). Q4 = hardest negatives (margin $<$ 0.02).

Quartile	Margin range	Emb Δ (pp)	DF Δ (pp)
Q1 (easiest)	> 0.15	+0.3	−0.2
Q2	0.08–0.15	+0.8	+0.1
Q3	0.02–0.08	+1.4	+0.5
Q4 (hardest)	< 0.02	+1.6	+1.2

content—neither mining strategy addresses this. Structural gap cases (28%) are where dataflow helps most (57% of such cases), but this ceiling limits overall impact.

Illustrative Case Studies. Table 19 presents 7 selected cases where strategies changed retrieval outcome. These are chosen to illustrate the mechanism, not as representative evidence—overall effect sizes are modest (Tables 1–3). Dataflow uniquely helps in cases involving cross-module callees, config propagation, and dependency cycles, where structural connectivity provides signal orthogonal to text similarity.

4.8 Comparison with Inference-Time Graph Methods

Table 20 positions our approach relative to inference-time graph methods. The key distinction is that these methods navigate structure at inference time, while we use structure for training-time data curation. The two approaches are in principle complementary—a retriever trained with structure-grounded negatives could serve as the backbone for an inference-time graph navigator—but we have not experimentally verified this complementarity.

5 Discussion

Summary of Findings. True embedding-distance mining tends to improve contrastive code retrieval over random baselines, with consistent direction across two encoders, two benchmarks, multiple seeds, and a range of hyperparameters. However, effect sizes are modest (0.5–1.0 pp R@1 in most conditions, up to 1.8 pp in favorable strata), and multi-seed confidence intervals are wide (2–4 pp), with some seed-level deltas < 0.5 pp. BM25-proxy negatives trend toward harm, consistent with the hypothesis that CoCoHaNeRe's reported −27 pp was amplified by proxy conflation, but our BM25 effect is not statistically significant ($p > 0.05$).

Dataflow as Exploratory Observation. Dataflow-grounded negatives show an encoder–strategy interaction: marginal positive effect on Qwen (CI includes 0) and negative effect on StarCoder. This is consistent with a representation saturation hypothesis—when the encoder's pretrained space has high positive similarity (Qwen: pos_sim = 0.99), structural negatives may provide an orthogonal

Table 9: R@1 by pool size (Qwen). Embedding advantage is consistent but diminishes at larger pools.

Pool	Random	Embedding	Dataflow	Emb Δ (pp)	Ranking stable?
50	0.530	0.540	0.532	+1.0	✓
100	0.412	0.421	0.413	+0.9	✓
200	0.328	0.335	0.330	+0.7	✓
500	0.241	0.246	0.244	+0.5	✓

Table 10: R@1 by pool size (StarCoder). At pool ≥ 200, strategy differences converge.

Pool	Random	Embedding	Dataflow	Emb Δ (pp)	Stable?
50	0.443	0.450	0.437	+0.7	✓
100	0.339	0.345	0.334	+0.6	✓
200	0.268	0.272	0.271	+0.4	$\sim$
500	0.194	0.196	0.197	+0.2	$\times$

signal dimension. However, the evidence is insufficient for an independent claim, and we present this as an exploratory observation that merits further investigation with more encoders.

Where Mining Matters. Stratified analysis reveals that mining benefit concentrates in specific conditions: small repositories, hard negative difficulty quartiles, and medium query-positive similarity ranges. These are precisely the conditions where the encoder's ability to discriminate is most challenged. However, these favorable conditions represent a minority of real-world instances, diluting the overall effect.

Practical Implications. For practitioners choosing a negative mining strategy for code retrieval: (1) embedding-distance mining is a reasonable default that tends to help and is unlikely to harm; (2) avoid using BM25 top-k as a proxy for embedding-distance negatives; (3) structural negatives via dataflow are not recommended as a general-purpose strategy given limited coverage (34%), encoder-dependent effects, and modest overall benefit.

Limitations.

- **Two encoders**: Our interaction finding rests on $n = 2$ encoders. Generalization requires broader evaluation.
- **Pool size**: Training uses pool=50; at realistic pool sizes (200–1000+), strategy advantages shrink further (Table 9).
- **Structural coverage**: Dataflow data is available for only 34% of instances, diluting any structural benefit.
- **SWE-bench power**: With $N = 300$ binary outcomes, SWE-bench is underpowered to detect retrieval-stage deltas < 2 pp.
- **Single training domain**: All training uses SWE-smith; domain shift to other retrieval tasks may alter findings.

Table 11: R@1 by temperature (Qwen). Strategy ranking is preserved; delta shrinks at higher τ.

τ	Random	Embedding	Dataflow	Emb Δ (pp)
0.05	0.530	0.540	0.532	+1.0
0.10	0.537	0.545	0.537	+0.8
0.20	0.521	0.526	0.520	+0.5

Table 12: R@1 by temperature (StarCoder). Same pattern: ranking preserved, delta shrinks.

τ	Random	Embedding	Dataflow	Emb Δ (pp)
0.05	0.443	0.450	0.437	+0.7
0.10	0.451	0.457	0.446	+0.6
0.20	0.435	0.438	0.430	+0.3

- **Effect magnitude**: All observed deltas are ≤ 2 pp, which may be within the noise floor for practical deployment decisions.

6 Conclusion

We presented a systematic study of hard negative mining strategies for contrastive code retrieval, comparing random, BM25-proxy, embedding-distance, and dataflow-disconnected negatives across two encoders, two benchmarks, and extensive ablations. Our results suggest that true embedding-distance mining tends to help and that prior reports of harm were likely confounded by BM25-proxy conflation, though the evidence for the latter is directional rather than conclusive. Structure-grounded negatives show an intriguing encoder-dependent interaction but do not provide consistent benefit and remain an exploratory observation. Stratified analysis reveals that mining benefit concentrates in specific conditions, providing a representation geometry explanation for the heterogeneous findings in the literature. We hope this systematic analysis helps the community make more informed choices about negative mining strategies and motivates further investigation into the interaction between encoder properties and training data curation.

References

V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. Bennett, J. Ahmed, and A. Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *ICLR*, 2021.

Table 13: R@1 by training data size (Qwen). Embedding advantage grows with data, sub-linearly.

Size	Random	Embedding	Dataflow	Emb Δ (pp)
1K	0.498	0.501	0.495	+0.3
3K	0.519	0.528	0.521	+0.9
5K (full)	0.530	0.540	0.532	+1.0

Table 14: Fusion head ablation. Structural features alone are weak; combining with embeddings adds marginal value.

Encoder	Emb-only	Struct-only	Combined	Combined−Emb (pp)
Qwen	0.539	0.412	0.541	+0.2
StarCoder	0.449	0.378	0.451	+0.2

Table 15: Dataflow R@1 by hop count and negative pool statistics.

	R@1		Pool statistics	
Hops	Qwen	StarCoder	Mean neg/query	Fallback %
1	0.530	0.434	3.2	42%
2	0.532	0.437	8.7	28%
3	0.529	0.436	14.1	18%

Table 16: Static vs. dynamic embedding mining (Qwen). Dynamic adds negligible benefit at 2.3× cost.

Mining	R@1	Δ vs static (pp)	Train time (h)	95% CI
Static	0.540	—	7.5	—
Dynamic	0.544	+0.4	17.3	[−0.8, 1.6]

Table 17: Binary vs. weighted dataflow negatives. No significant difference.

Encoder	Binary DF R@1	Weighted DF R@1	Δ (pp)
Qwen	0.532	0.535	+0.3
StarCoder	0.437	0.439	+0.2

Table 18: Failure categories from 50 SWE-bench retrieval failures, with per-strategy help rates.

Category	Count (%)	DF helps	Emb helps
Structural gap	14 (28%)	8 (57%)	5 (36%)
Lexical gap	18 (36%)	3 (17%)	6 (33%)
Multi-file dep.	8 (16%)	4 (50%)	5 (63%)
Other	10 (20%)	1 (10%)	2 (20%)

Table 19: Selected qualitative cases. Ranks are within the candidate pool. Cases are illustrative, not representative.

#	Issue type	Rand	Emb	DF	Key observation
1	Missing import	12	8	2	DF finds cross-module callee
2	API mismatch	9	3	4	Emb finds similar naming
3	Config propagation	15	11	1	DF traces config flow
4	Type error	6	2	3	Both find related types
5	Null pointer	11	7	3	DF finds data source
6	Method override	4	1	5	Emb captures signature sim
7	Circular dep	14	10	2	DF traces dependency cycle

Table 20: Comparison with inference-time graph methods. StructFuse operates at training time, making it complementary.

Method	Graph type	When used	Complementary?
CodexGraph	Repository graph	Inference	Yes
LocAgent	Code structure	Inference	Yes
RANGER	AST + deps	Inference	Yes
StructFuse (ours)	Dataflow	Training	—